МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМ. Р.Е. АЛЕКСЕЕВА»
(НГТУ)

Кафедра «Компьютерные технологии в проектировании и производстве»

Заведующий кафедрой

_____ Моругин С.Л.
(*подпись*) (*фамилия, и., о.*)

_____
(*дата*)

Разработка информационной системы «Салон красоты»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе

КОНСУЛЬТАНТЫ:

1.По _____

_____ _____
(*подпись*) (*фамилия, и., о.*)

_____
(*дата*)

2.По _____

_____ _____
(*подпись*) (*фамилия, и., о.*)

_____
(*дата*)

3.По _____

_____ _____
(*подпись*) (*фамилия, и., о.*)

_____
(*дата*)

РЕЦЕНЗЕНТ

_____ _____
(*подпись*) (*фамилия, и., о.*)

_____
(*дата*)

РУКОВОДИТЕЛЬ:

_____ Штанюк А.А.
(*подпись*) (*фамилия, и., о.*)

_____
(*дата*)

СТУДЕНТ:

_____ Солодун М.М.
(*подпись*) (*фамилия, и., о.*)

_____ 23-ИСТ-1-1
(*дата*) (*группа или шифр*)

Проект защищен _____

Протокол № _____

С оценкой_____

2024

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ им. Р.Е. Алексеева»
(НГТУ)

Кафедра Компьютерные технологии в проектировании и производстве

УТВЕРЖДАЮ
Зав. кафедрой
_____С.Л. Моругин
«__»_____2021 г.

ЗАДАНИЕ

на курсовую работу
(бакалавриата/специалитета/магистратуры)

по направлению 090302 – Информационные системы и технологии
(код и наименование)

Студенту Солодуну Максиму Михайловичу группы 23-ИСТ-1-1
(ФИО)

1. Тема КП: Разработка информационной системы «Салон красоты»
2. Срок сдачи студентом законченной работы: 19.12.2024
3. Исходные данные к работе: Система должна обеспечивать: 1) возможность управления системой из главного меню 2) возможность просмотра списка клиентов салона 3) возможность добавления/удаления клиента 4) возможность просмотра списка мастеров салона 5) возможность добавления/удаления мастера 6) возможность просмотра списка всех записей 7) возможность добавления/удаления записи.
4. Содержание расчетно-пояснительной записки (перечень вопросов, подлежащих разработке):

- Введение;
- Разработка и анализ технического задания

- Разработка модели предметной области;
- Разработка информационной системы;
- Описание комплекта тестов;
- Руководство пользователя
- Заключение
- Список литературы;
- Приложения.

5. Перечень графического материала (с точным указанием обязательных чертежей):

- Диаграммы модели предметной области (диаграммы классов UML);
- Диаграммы, описывающие архитектуру системы (диаграммы классов UML);
- Диаграммы последовательностей системы;
- Результаты выполнения тестового комплекта.

6. Консультанты по работе (с указанием относящихся к ним разделов проекта):

_____

_____

7. Дата выдачи задания: 20.09.2024

Руководитель: _____ А.А. Штанюк
(*подпись*)

Задание принял к исполнению _____
(*дата*)

Студент: _____ М.М. Солодун
(*подпись*)

Содержание

1. Введение 2

2. Разработка и анализ ТЗ 4

2.1. Исследование предметной области 4

2.2. Функциональные требования 5

2.3. Требования к интерфейсу 6

2.4. Выбор и обоснование средств хранения данных.....8

3. Разработка ИС 10

3.1. Анализ функционала и структуры аналога разрабатываемой системы10

3.2. Модель предметной области 12

3.3. CRC карточки классов системы 15

3.4. Диаграмма классов системы 17

3.5. Взаимодействие объектов 18

3.6. Диаграмма деятельности 19

3.7. Модель данных 20

3.8. Список шаблонов и принципов ООП, использованных в работе …21

3.9. Тесты 22

4. Руководство пользователя 24

5. Заключение 29

Список литературы 30

Приложение 31

1. Введение

Современная индустрия красоты характеризуется высокой конкуренцией и стремительно растущими ожиданиями клиентов. Для обеспечения эффективного управления и оптимизации рабочих процессов салоны красоты все чаще обращаются к информационным технологиям. Разработка исследовательской системы для салона красоты является актуальным шагом в повышении производительности работы персонала, улучшении качества услуг и минимизации организационных ошибок.

Данная исследовательская система предназначена исключительно для внутреннего использования персоналом салона и обеспечивает автоматизацию следующих ключевых функций:

- **Управление клиентами:** добавление новых клиентов в базу данных с указанием контактной информации и удаление устаревших записей. Это позволяет вести актуальный учет клиентской базы и упрощает планирование.
- **Управление мастерами:** добавление сотрудников с указанием их специализации (парикмахер, маникюрист, педикюрист) и удаление записей мастеров, которые больше не работают в салоне.
- **Работа с журналом записей:** запись клиентов на процедуры с учетом расписания мастеров и их специализации, а также удаление записей при необходимости.

Система ориентирована на оптимизацию работы персонала, предоставляя удобные инструменты для планирования и учета. Клиенты не взаимодействуют напрямую с данной системой, что гарантирует персоналу полный контроль над процессами.

Проект разработан с применением принципов объектно-ориентированного программирования (ООП), которые играют ключевую роль в создании

надежного и легко модифицируемого программного обеспечения. Использование ООП позволяет разбить программу на классы и их объекты, что обеспечивает модульность, повторное использование кода, упрощение разработки и поддержки, а также гибкость и масштабируемость систем.

Таким образом, исследовательская система становится важным инструментом для повышения эффективности работы персонала салона. Она минимизирует влияние человеческого фактора, упрощает управление клиентской базой, расписанием и составом сотрудников. Кроме того, проект остается гибким и масштабируемым, позволяя в будущем добавлять новые функции и адаптироваться к изменяющимся требованиям бизнеса.

2. Разработка и анализ ТЗ

2.1. Исследование предметной области

Салон красоты представляет собой организацию, предоставляющую широкий спектр услуг по уходу за внешностью, включая парикмахерские услуги, маникюр, педикюр, косметологические процедуры и другие. Успешное функционирование салона требует систематизации работы персонала, учета клиентов, а также эффективного управления записями и услугами.

Основными участниками процессов являются мастера (парикмахеры, маникюристы, педикюристы и другие специалисты), администраторы и клиенты. В обязанности администратора входит запись клиентов на процедуры, управление расписанием мастеров, а также ведение базы данных клиентов и их историй посещений. При этом важно учитывать следующие аспекты:

1. **Учет клиентов**: Для улучшения обслуживания необходимо хранить данные о клиентах, включая их имя, контакты и историю записей на процедуры. Это помогает персонализировать подход и выстраивать долгосрочные отношения.
2. **Учет мастеров и их специализаций**: Каждый мастер имеет определенные навыки и предоставляет конкретные виды услуг. Необходимо учитывать графики работы мастеров, чтобы эффективно распределять рабочую нагрузку.
3. **Ведение журнала записей**: Своевременная запись клиентов и управление расписанием процедур позволяет оптимизировать занятость мастеров и минимизировать простои.

4. **Автоматизация процесса**: Использование программных решений позволяет сократить рутинные операции, уменьшить вероятность ошибок и повысить общую эффективность работы салона.

Разработка исследовательской системы для салона красоты основывается на необходимости автоматизации вышеуказанных процессов. Целью такой системы является создание удобного инструмента для администратора, который упрощает учет клиентов, управление расписанием мастеров и организацию записей на процедуры, способствуя повышению качества услуг и эффективности работы салона.

2.2. Функциональные требования

Информационная система для салона красоты должна обеспечивать следующие функциональные возможности, ориентируясь на потребности персонала и специфику работы салона:

1. **Управление данными клиентов**:
   - Добавление нового клиента с указанием его имени, фамилии, контактных данных и другой необходимой информации.
   - Удаление клиента из системы по его уникальному идентификатору (ID).
   - Ведение списка клиентов с отображением их общей информации, включая количество записей на процедуры.
2. **Управление данными мастеров**:
   - Добавление нового мастера с указанием его специализации (парикмахер, маникюрист, педикюрист и т. д.), контактных данных и списка предоставляемых услуг.
   - Удаление мастера из системы по его уникальному идентификатору (ID).

- Ведение списка мастеров с отображением их специализаций, контактной информации и других данных.

3. **Управление записями на процедуры**:
   - Создание новой записи для клиента, включающей выбор мастера, тип процедуры, дату и время.
   - Удаление записи из журнала по ее уникальному идентификатору (ID).
   - Отображение журнала записей.
4. **Интеграция с базой данных**:
   - Сохранение данных о клиентах, мастерах и записях в формате JSON-файлов для обеспечения их надежности и возможности последующей загрузки.
   - Автоматическая загрузка данных из базы при запуске системы, включая информацию о клиентах, мастерах и существующих записях.

5. **Закрытие системы с сохранением данных**:

   - При закрытии главной формы данные всех коллекций (клиентов, мастеров, записей) должны автоматически сохраняться в базе данных.

Эти функциональные требования направлены на обеспечение удобства использования системы и автоматизацию основных бизнес-процессов салона красоты.

2.3. Требование к интерфейсу

Интерфейс информационной системы для салона красоты должен быть интуитивно понятным, эргономичным и функциональным, чтобы персонал салона мог эффективно использовать систему без предварительного обучения. Основные требования к интерфейсу:

1. **<u>Простота и удобство использования</u>**:
    - Все основные функции системы (управление клиентами, мастерами и записями) должны быть доступны через главную форму.
    - Переход между формами должен осуществляться с помощью удобных и наглядных кнопок.
2. **<u>Отображение данных</u>**:
    - Для отображения списков клиентов, мастеров и записей должны использоваться таблицы (DataGridView), предоставляющие структурированный вид данных.
    - Каждая таблица должна содержать соответствующие столбцы:
        - **Для клиентов**: ID, имя, фамилия, телефон, почта, количество записей.
        - **Для мастеров**: ID, имя, фамилия, телефон, почта, тип мастера.
        - **Для записей**: ID записи, ID клиента, дата и время записи, тип услуги.
3. **<u>Функциональные кнопки</u>**:
    - На каждой форме должны быть кнопки для выполнения ключевых операций:
        - Добавление новых данных (клиента, мастера или записи).
        - Удаление существующих данных с использованием ID.
        - Кнопка "Выйти" для возврата на главную форму.

4. **<u>Скорость и отзывчивость приложения:</u>**

    - Интерфейс должен загружаться быстро и действия пользователя должны вызывать мгновенную реакцию.

2.4. Выбор и обоснование средств хранения данных

Для хранения данных в информационной системе салона красоты выбран формат JSON (JavaScript Object Notation). Этот выбор обусловлен несколькими ключевыми факторами:

1. **Простота и читабельность**:
   - Формат JSON обладает удобной структурой, представленной в виде пар "ключ-значение", что делает данные легко читаемыми как для человека, так и для программы.
   - Данные, хранящиеся в JSON, можно быстро анализировать и редактировать при необходимости, что полезно для тестирования и отладки системы.
2. **Легкость интеграции с .NET Framework**:
   - В проекте используется библиотека **Newtonsoft.Json**, которая обеспечивает удобные методы сериализации и десериализации данных. Это позволяет эффективно преобразовывать объекты классов в JSON и обратно, минимизируя затраты на разработку.
3. **Отсутствие сложной инфраструктуры**:
   - Для небольших систем, таких как данная информационная система, использование полноценной базы данных может быть избыточным. JSON-файлы не требуют установки серверов или дополнительных библиотек, что упрощает развертывание и эксплуатацию.

Использование JSON как основного средства хранения данных обеспечивает баланс между простотой реализации, удобством разработки и эксплуатацией системы. Такой подход позволяет сосредоточиться на логике приложения и обеспечивать поддержку всех необходимых функций без сложных затрат на управление базами данных.

При дальнейшем развитии системы возможен переход на более сложные технологии хранения данных, такие как реляционные базы данных (SQL) или облачные решения, для обеспечения работы с большими объемами данных или многопользовательского доступа.

3. Разработка ИС

3.1. Анализ функционала и структуры аналога разрабатываемой системы

Для успешной реализации информационной системы для салона красоты важно провести анализ существующих решений в данной области. Одним из примеров является веб-приложение сайта центра косметологии Liberty[1], который предоставляет услуги клиентам и способствует оптимизации записи на процедуры. Это позволяет оценить сильные и слабые стороны такого аналога, чтобы учесть их при разработке собственной системы, ориентированной на внутреннее управление процессами в салоне красоты.

Функционал сайта

Веб-приложение предоставляет следующие возможности:

1) Просмотр перечня услуг: Раздел с услугами содержит полное описание процедур, включая их стоимость, продолжительность, методику проведения и ожидаемый эффект. Это позволяет клиентам получить подробное представление о предлагаемых услугах и выбрать подходящую процедуру.

2) Данные о специалистах: Указание квалификаций и опыта работы специалистов помогает клиентам сделать осознанный выбор. Для каждого мастера приводится их специализация, что упрощает процесс подбора.

3) Функционал онлайн-записи: Система онлайн-записи предоставляет удобный интерфейс для выбора мастера, даты и времени процедуры. Это уменьшает временные затраты как для клиентов, так и для персонала.

4) Информация об акциях и скидках: Раздел с акциями мотивирует клиентов воспользоваться услугами центра косметологии, что положительно сказывается на привлечении новых клиентов.

5) Контактные данные и навигация: На сайте представлены телефоны, адрес, а также карта расположения салона. Эти данные помогают клиентам оперативно получить нужную информацию.

К преимуществам данного приложения можно отнести:

- Дружественный интерфейс: Простота навигации делает сайт интуитивно понятным даже для новых пользователей. Адаптивный дизайн позволяет использовать его как на стационарных компьютерах, так и на мобильных устройствах.
- Широкий функционал для клиентов: Сайт охватывает основные запросы целевой аудитории: выбор процедур, запись на услуги, а также поиск информации о мастерах и акциях.
- Современный визуальный стиль: Дизайн сайта отвечает современным стандартам, что положительно влияет на восприятие бренда.

К недостаткам данного приложения отнесём:

- Ограниченная направленность на внешнего пользователя: Функционал ориентирован исключительно на клиентов, что оставляет без внимания потребности сотрудников салона. Нет возможности для внутреннего учета данных о клиентах, мастерах, а также записей.
- Отсутствие персонализации: Пользовательские профили не представлены. Это ограничивает возможности сохранения истории услуг или настройки интерфейса под предпочтения клиента.
- Недостаточный функционал для аналитики и управления: Приложение не предоставляет инструментов для анализа статистики, планирования работы сотрудников или управления расписанием.

Анализ веб-приложения Liberty показывает, что оно эффективно решает задачи внешнего взаимодействия с клиентами. Однако разрабатываемая информационная система для салона красоты должна быть реализована с функционалом, ориентированным на автоматизацию внутренних процессов. Это включает: ведение базы данных клиентов, мастеров и записей, управление графиками мастеров и процедур, учет и аналитика загруженности салона для повышения эффективности его работы.

Интеграция такого функционала в информационную систему позволит оптимизировать рабочие процессы персонала, минимизировать ручной труд и повысить уровень обслуживания клиентов.

Приложение Liberty может послужить хорошим пример для разработки информационной системы салона красоты. Опираясь на все его сильные и слабые можно создать функциональную и удобную систему для пользователя по направлению салона красоты.

3.2. Модель предметной области

Модель предметной области разрабатываемой информационной системы для салона красоты представляет собой совокупность объектов и их взаимосвязей, отражающих основные процессы и сущности, характерные для данной сферы. Основной задачей модели является формализация структуры данных и процессов, что позволит эффективно автоматизировать работу персонала.

В качестве объектов предметной области салона красоты выступают: клиент, мастер и запись. Объект клиент содержит информацию о: id, имени, фамилии, номере телефона и почтовом адресе клиента. Объект мастер содержит информацию о: id, имени, фамилии, номере телефона, почтовом адресе и типе данного мастера. Объект запись содержит информацию о: id записи, id клиента, id мастера, времени процедуры, типы процедуры и цене.

Чтобы выявить взаимосвязи объектов в данной предметной области следует выделить основные сущности и создать модель предметной области, рисунок 1.

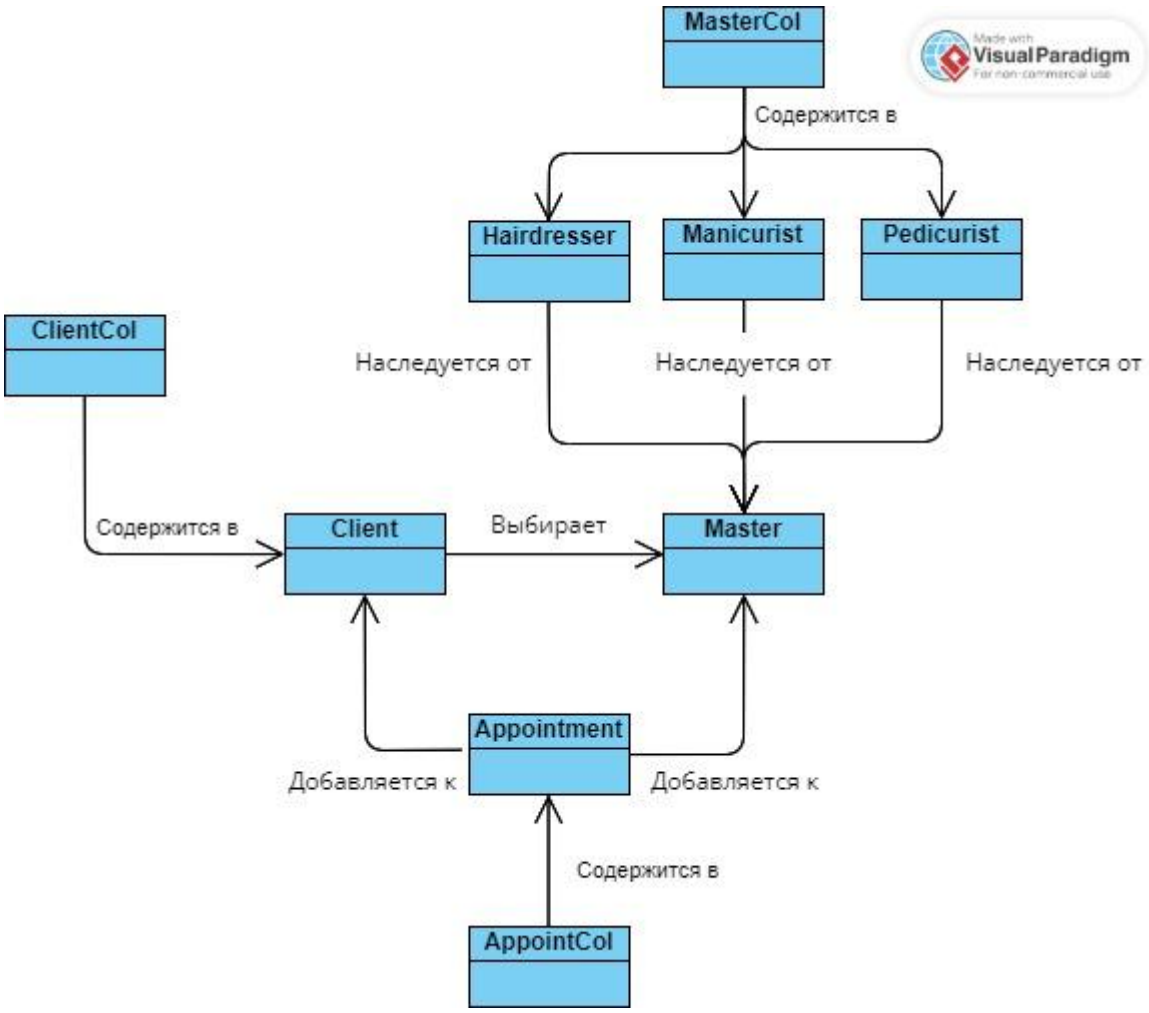

Рисунок 1 – Модель предметной области ИС.

Исходя из модели предметной области можно сделать краткое описание модели:

1. **ClientCol (Коллекция клиентов):**
   - Это контейнер или коллекция, которая хранит все объекты **Client** (клиенты). Коллекция управляет клиентами в системе.
2. **Client (Клиент):**

- Клиент является важной сущностью в системе. Он может выбирать мастера для записи на услугу. Каждый клиент будет иметь атрибуты, такие как имя, фамилия и контактные данные.

3. **MasterCol (Коллекция мастеров):**
   - Это контейнер или коллекция мастеров, которая хранит все объекты **Master** (мастера) в системе. Коллекция позволяет системе управлять мастерами и их данными.
4. **Master (Мастер):**
   - Мастер - это сущность, представляющая профессионала в салоне. Мастер может быть разных типов, например:
     - **Hairdresser (Парикмахер)**
     - **Manicurist (Маникюрист)**
     - **Pedicurist (Педикюрист)**
   - Эти типы мастеров наследуют основные свойства сущности **Master**, что отражает иерархическую структуру, где каждый мастер может быть специализирован в определенной области.
5. **Appointment (Запись):**
   - Запись представляет собой процесс, когда клиент записывается к мастеру на услугу. Эта сущность хранит информацию о том, какой клиент выбрал какого мастера, а также детали записи (например, дата, время, услуга и цена). Записи связаны с клиентами и мастерами.
6. **AppointCol (Коллекция записей):**
   - Это коллекция, которая хранит все записи, сделанные клиентами для получения услуг от мастеров. Она позволяет управлять всеми записями в системе.

Модель отражает типичную структуру информационной системы для салона красоты, где клиенты могут записываться на различные услуги (например, стрижку, маникюр, педикюр) через мастеров.

3.3. CRC карточки классов системы

Для успешного создания хорошой информационной системы следует создать CRC-карточки данной системы.

CRC-карточки (Class-Responsibility-Collaboration cards) — это инструмент, используемый в объектно-ориентированном проектировании для определения и описания классов в системе, а также их ответственности и взаимодействия с другими классами.

CRC-карточки информационной системы для салон красоты представлены на рисунках ниже, рисунок 2,3,4.

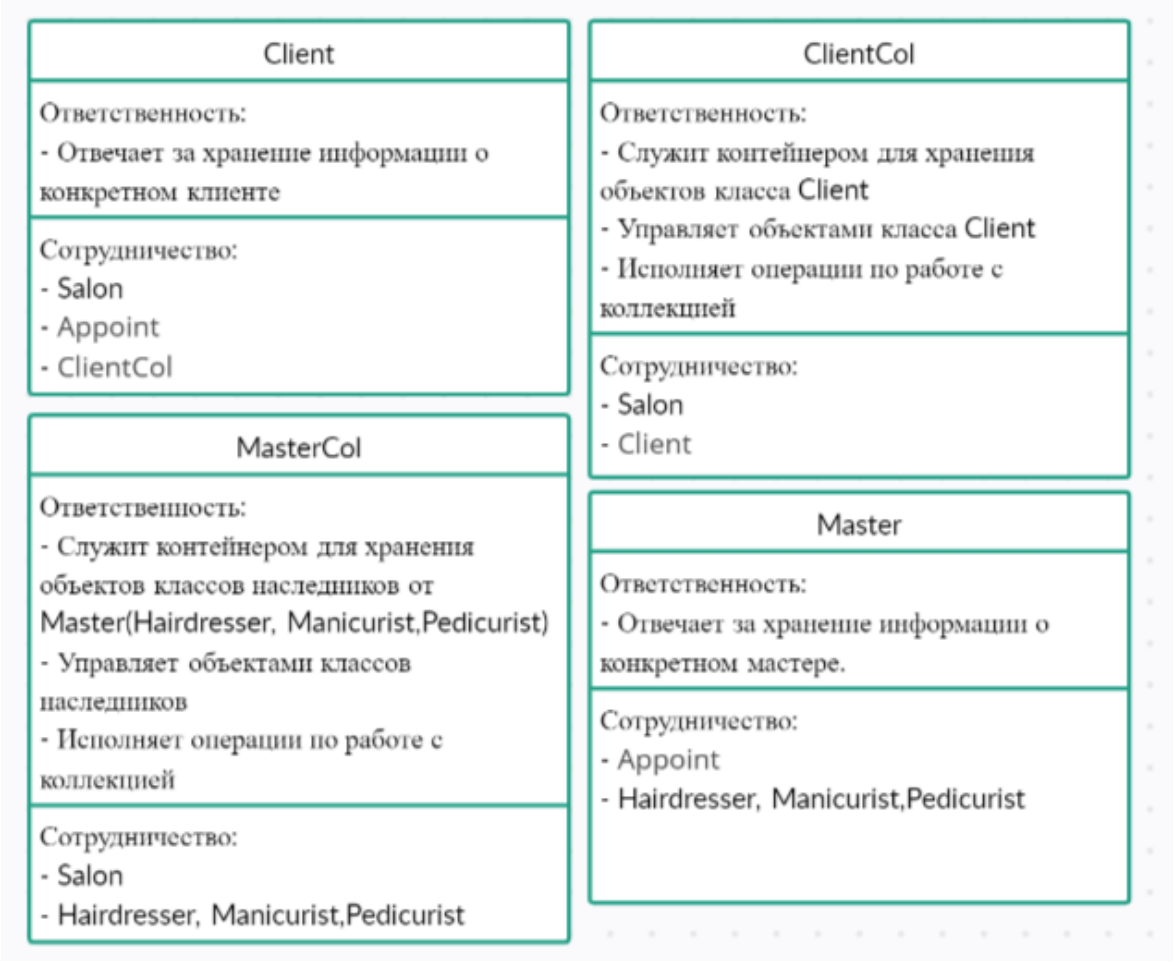

Рисунок 2 – CRC-карточки информационной системы для салона красоты.

Рисунок 3 - CRC-карточки информационной системы для салона красоты.

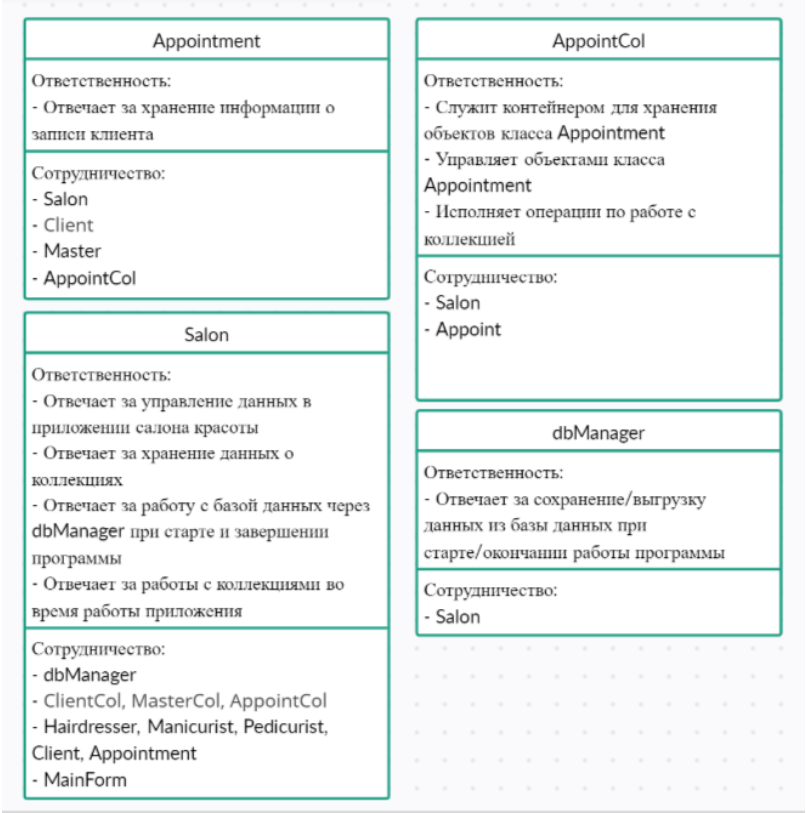

Рисунок 4 - CRC-карточки информационной системы для салона красоты.

3.4. Диаграмма классов системы

После создания CRC-карточек в соответствии с ними определим главную диаграмму классов, рисунок 5.

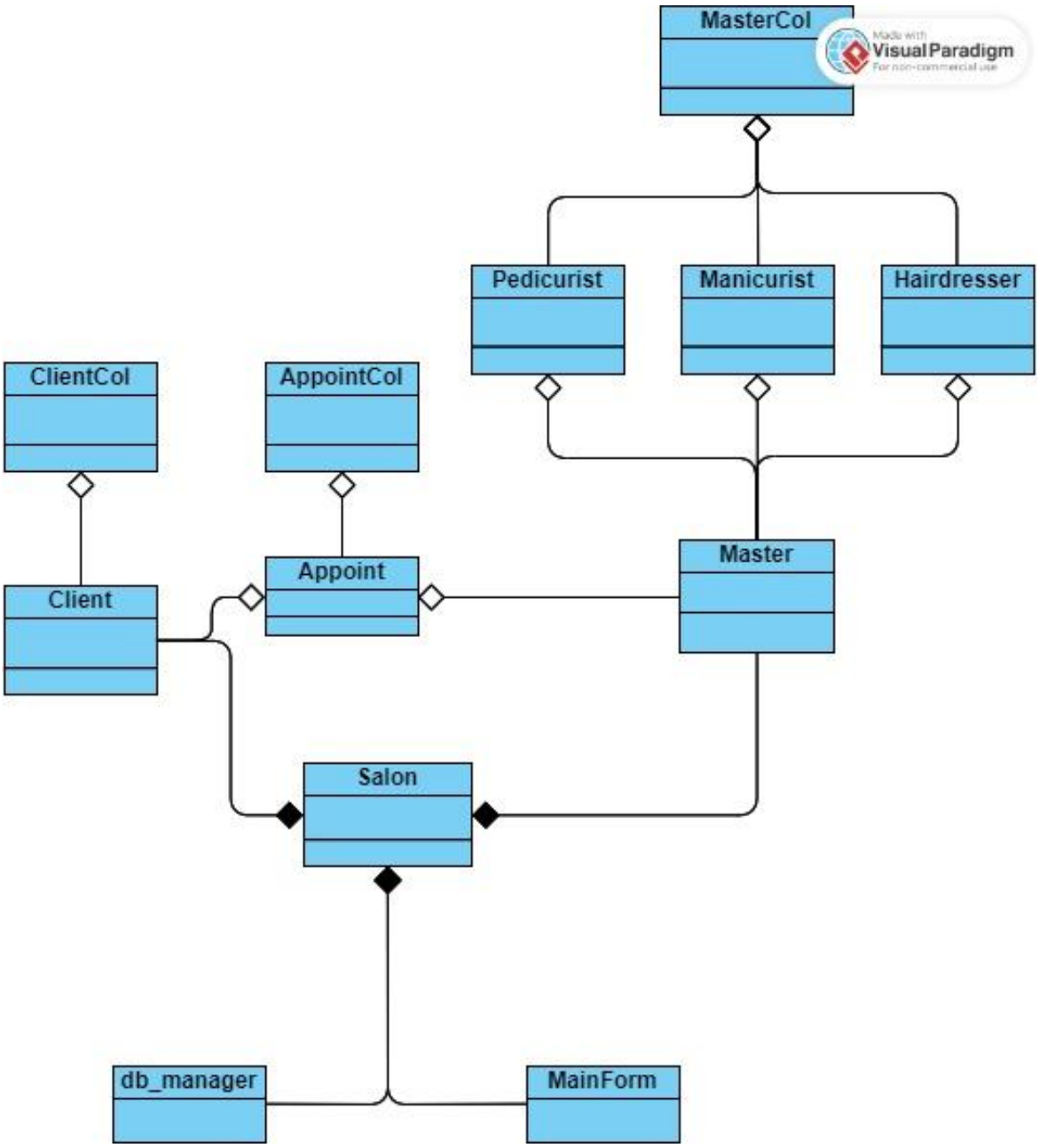

Рисунок 5 – Диаграмма классов системы.

3.5. Взаимодействие объектов

Диаграммы последовательности — это вид диаграмм взаимодействия в UML (Unified Modeling Language), предназначенный для моделирования обмена сообщениями между объектами или компонентами системы. Они отображают, как элементы взаимодействуют друг с другом для выполнения определенной функциональности, фокусируясь на временном порядке отправки и получения сообщений. Эти диаграммы полезны для описания сценариев использования, проектирования алгоритмов и выявления потенциальных проблем в логике взаимодействия.

Диаграмма состоит из вертикальных линий, представляющих объекты или участники взаимодействия, и горизонтальных стрелок, обозначающих сообщения. Временная шкала идёт слева направо, показывая последовательность операций. Сообщения могут быть синхронными (требующими ответа) или асинхронными.

На диаграмме снизу представлена диаграмма последовательностей добавления мастера, рисунок 6.

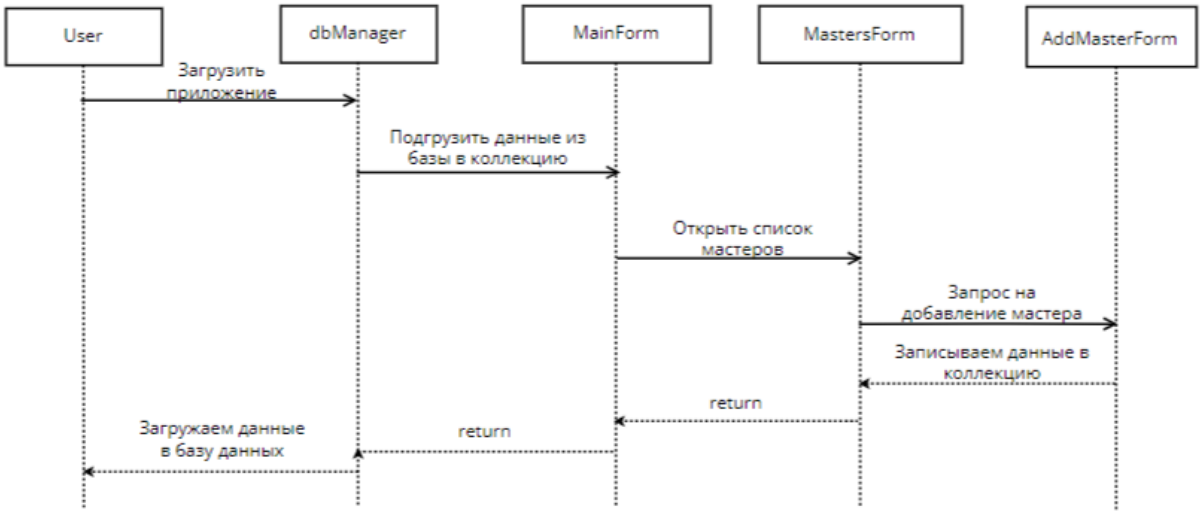

Рисунок 6 – Диаграмма последовательностей добавления мастера.

3.6. Диаграмма деятельности

Диаграммы деятельности — это инструмент в UML, который используется для моделирования процессов и рабочих потоков в системе. Они отображают последовательность действий, выполняемых для достижения определенной цели, и помогают визуализировать поведение системы на высоком уровне. Такие диаграммы применяются для описания бизнес-процессов, алгоритмов и логики выполнения операций, обеспечивая наглядность и облегчая понимание сложных процессов.

На диаграмме снизу представлена диаграмма деятельности добавления записи, рисунок 6.

Рисунок 7 - Диаграмма деятельности добавления записи.

3.7. Модель данных

В качестве формата хранения данных в проекте информационной системы салона красоты используется формат JSON (JavaScript Object Notation). JSON представляет собой текстовый формат, предназначенный для хранения и обмена структурированной информацией. Он широко используется благодаря своей читаемости для человека и удобству обработки программами.

JSON-файлы в проекте служат хранилищем данных о клиентах, мастерах, и записях на услуги. Каждая сущность имеет свою структуру данных, записанную в виде пар "ключ-значение", рисунок 8. Например, для клиента хранятся такие данные, как ID, имя, фамилия и контактная информация. Для мастеров предусмотрены их тип (парикмахер, маникюрист или педикюрист), а также ID, имя, фамилия, контактная информация и перечень предоставляемых услуг. Записи на услуги содержат ID, информацию о клиенте, мастере, дате и времени и типе процедуры.

```
[
  {
    "Id": 1,
    "Name": "София",
    "Surname": "Кудрявцева",
    "Phone": "89969928193",
    "Email": "sofa@mail.ru"
  },
  {
    "Id": 2,
    "Name": "Арсений",
    "Surname": "Жуков",
    "Phone": "89991378745",
    "Email": "arsen@gmail.com"
  }
]
```

Рисунок 8 – Пример JSON-файла с информацией clients.json.

Выбор JSON обусловлен несколькими причинами. Во-первых, JSON позволяет легко структурировать и организовывать данные. Во-вторых, он поддерживается большинством языков программирования, включая C#, что упрощает его интеграцию в проект. Наконец, JSON-файлы компактны и их легко редактировать вручную при необходимости. В проекте данные загружаются из JSON-файлов при запуске системы и сохраняются обратно в них при завершении работы. Такая модель обеспечивает простоту реализации и достаточную производительность для задач системы.

3.8. Список шаблонов и принципов ООП, использованных в работе

Информационная система салона красоты написана в соответствии с шаблонами и принципами ООП. Ниже представлен их список.

Принципы:

- Не повторяйся (Don’t repeat yourself — DRY)
- Принцип единственной ответственности(Single-responsibility principle – SRP)
- Будь проще(Keep It Simple, Stupid - KISS)
- Инкапсуляция(Encapsulation)
- Полиморфизм(Polymorphism)
- Наследование(Inheritance)

Шаблоны:

- Data mapper
- Controller(dbManager)
- MVC(Model-View-Controller) – этот шаблон подразумевает, что приложение будет разделено на 3 основные части:модель, представление и контроллер.

3.9. Тесты

Тестирование программного обеспечения необходимо для обеспечения его качества, надежности и соответствия заданным требованиям. Оно позволяет выявить ошибки, дефекты или несоответствия на ранних этапах разработки, снижая риски появления проблем в процессе эксплуатации. Тестирование помогает убедиться, что программа работает корректно в различных сценариях, удовлетворяет потребности пользователей и соответствует ожиданиям заказчика. Кроме того, оно способствует повышению безопасности, производительности и стабильности приложения, что критически важно для успешного использования программного обеспечения.

Чтобы проверить корректность работы программы создали модульные тесты, рисунок 9.

Тестирование	Длительн...	Признаки	С...
Tests (5)	39 мс		
Tests (5)	39 мс		
UnitTest1 (5)	39 мс		
AddClient_ShouldAddCli...	1 мс		
AddMaster_ShouldAdd...	< 1 мс		
RemoveClient_ShouldRe...	1 мс		
RemoveMaster_ShouldR...	< 1 мс		
SaveData_ShouldSaveCli...	37 мс		

Рисунок 9 – Модульные тесты приложения.

Чтобы проверить правильно ли работают контроллеры, которые работают с базой данных JSON, добавим тест кейсы, рисунок 10,11.

Test ID: 1	
Test Tittle: addMastertoDB	
Setup: Имя мастера: Евгений Фамилия мастера: Васильев Номер телефона: 89991378745 Почта: evg@gmail.com Тип: Парикмахер	
Execute part:	
Procedure:	Expected result:
1) Запустить приложение 2) Открыть форму "Список мастеров" 3) Нажать на кнопку "Добавить" 4) Ввести данные 5) Проверить результаты, открыв базу данных	Имя мастера: Евгений Фамилия мастера: Васильев Номер телефона: 89991378745 Почта: evg@gmail.com Тип: Парикмахер
Result: Pass	

Test ID: 2	
Test Tittle: addClientoDB	
Setup: Имя клиента: Мария Фамилия клиента: Ромашева Номер телефона: 89969928193 Почта: maria@gmail.com	
Execute part:	
Procedure:	Expected result:
1) Запустить приложение 2) Открыть форму "Список клиентов" 3) Нажать на кнопку "Добавить" 4) Ввести данные 5) Проверить результаты, открыв базу данных	Имя клиента: Мария Фамилия клиента: Ромашева Номер телефона: 89969928193 Почта: maria@gmail.com
Result: Pass	

Рисунок 10 – Добавление мастера и клиента в базу данных.

Test ID: 3	
Test Tittle: addAppointtoDB	
Setup: ID Мастера: 1 ID Клиента: 1 Дата записи: 20.12.2024 11:30 Тип процедуры: Стрижка	
Execute part:	
Procedure:	Expected result:
1) Запустить приложение 2) Открыть форму "Список записей" 3) Нажать на кнопку "Добавить" 4) Ввести данные 5) Проверить результаты, открыв базу данных	ID Мастера: 1 ID Клиента: 1 Дата записи: 20.12.2024 11:30 Тип процедуры: Стрижка Стоимость процедуры: 1000
Result: Pass	

Рисунок 11 – Добавление записи в базу данных.

4. Руководство пользователя

При запуске приложения нас встречает меню, с которым пользователь может взаимодействовать, рисунок 12. На главное форме присутствует информация о салоне, к которой пользователь может обратиться в любое время. Так же на главной форме присутствуют кнопки, по которым пользователь может переходить на списки клиентов, мастеров и записей.

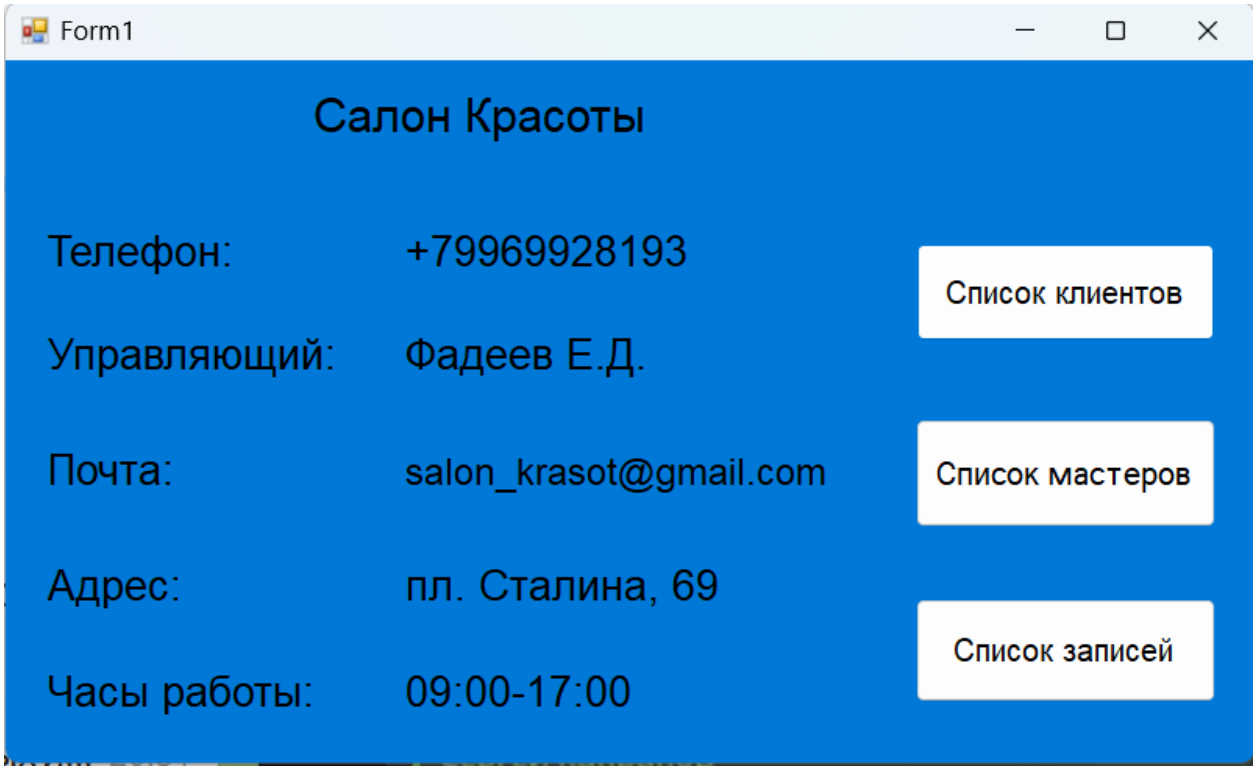

Рисунок 12 – Главная форма.

Далее если пользователь нажмёт на кнопку список клиентов, то ему откроется список всех клиентов салона красоты, зарегистрированных на данный момент, рисунок 13. После чего у пользователя есть возможность нажать на одну из трёх кнопок(Добавить, Удалить или Выйти). При нажатии на кнопку Добавить перед пользователем раскрывается ещё одна форма, где его просят ввести данные о клиенте салона красоты, а именно имя, фамилию и контактную информацию, а также присутствует кнопка «Добавить клиента» рисунок 14. Введя всю ифнормацию о клиенте и нажав кнопку добавить, при

достоверности всех данных, пользователь получает уведомление о том, что клиент добавлен, рисунок 15, а после чего может посмотреть уже изменённый список с добавленным клиентом, рисунок 16.

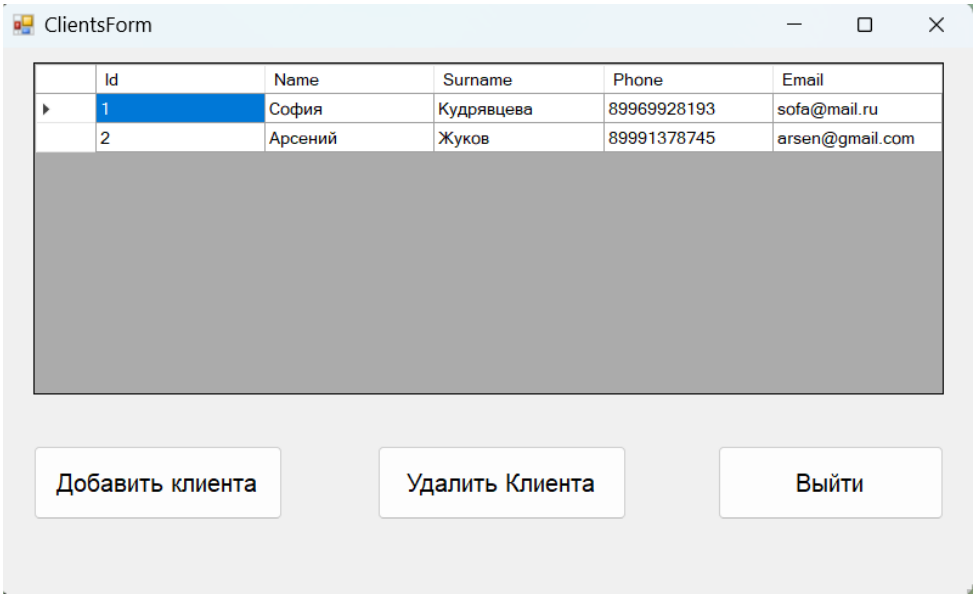

Рисунок 13 – Форма списка клиентов.

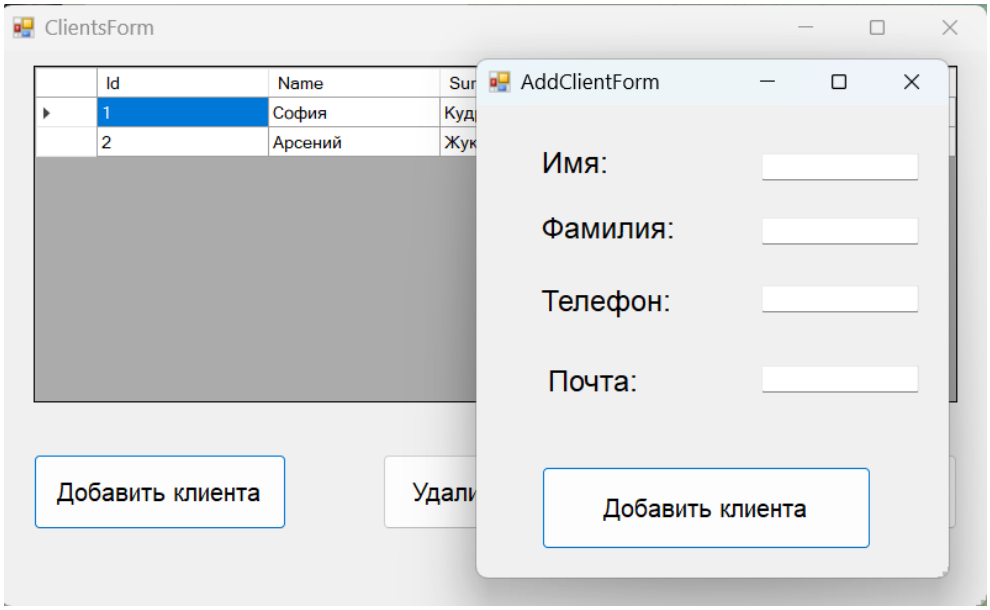

Рисунок 14 – Форма добавления клиента.

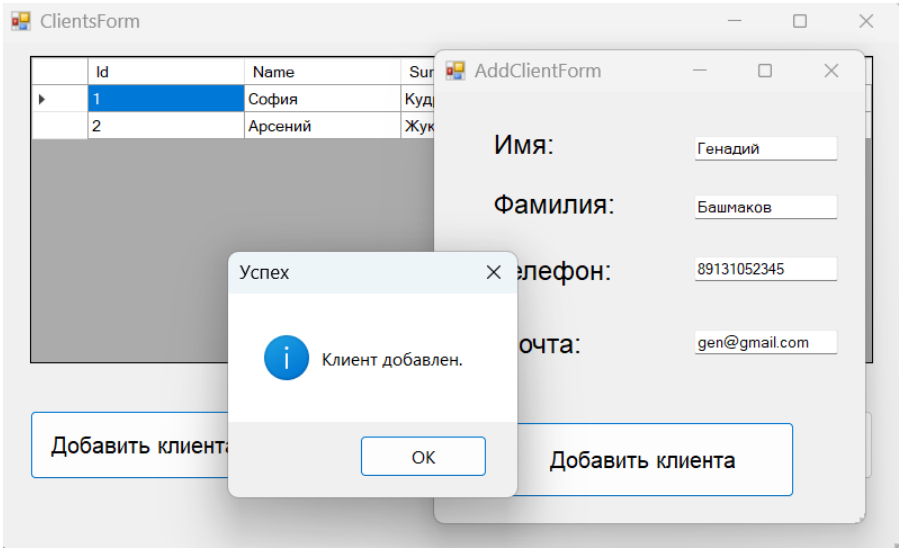

Рисунок 15 – Одобрение добавления клиента.

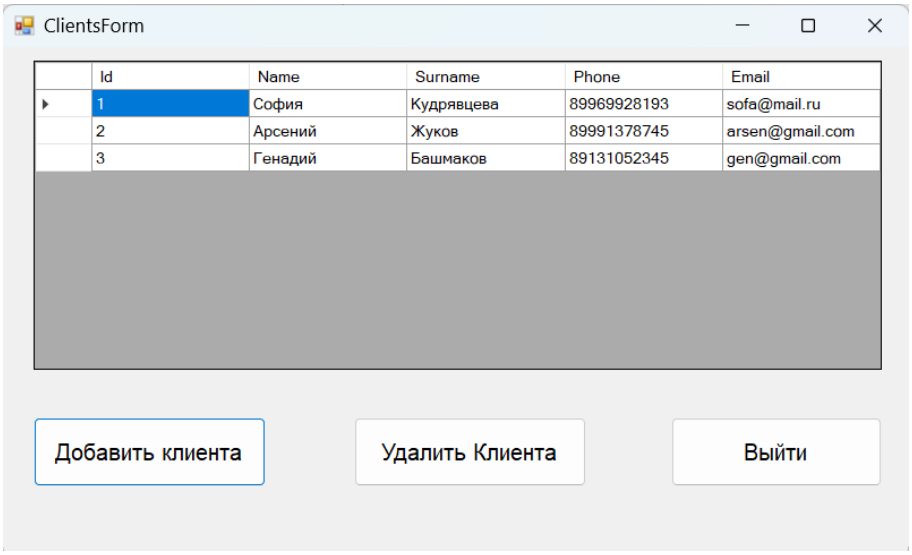

Рисунок 16 – Обновлённый список после добавления.

По кнопке «Удалить клиента» пользователь получает аналогичные формы, однако в это случае его просят ввести ID удаляемого клиента, рисунок 17. После данных операций клиент получает опять же обновлённую таблицу, рисунок 18.

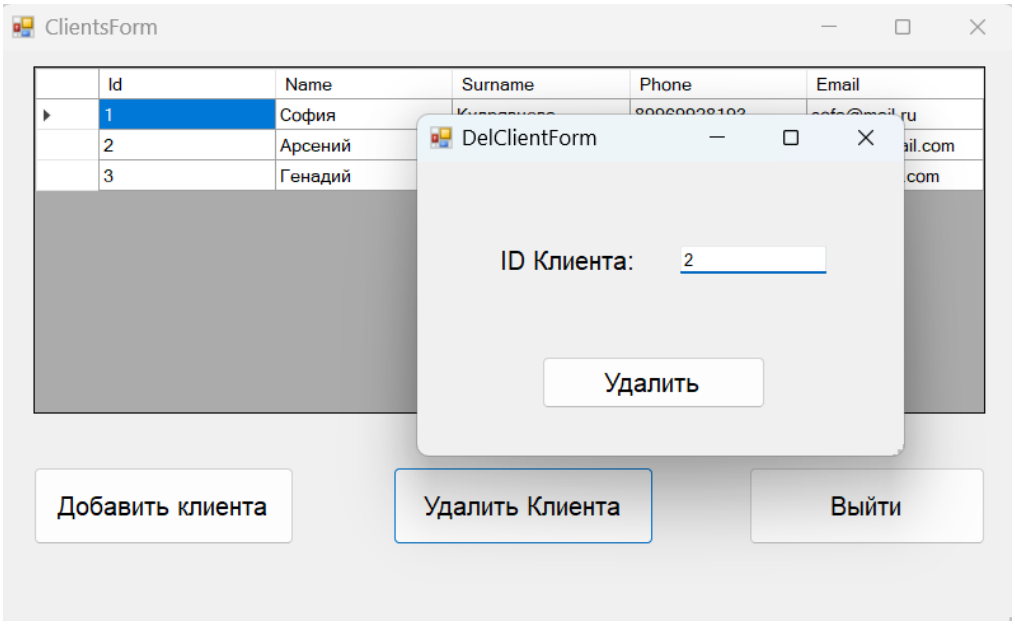

Рисунок 17 – Удаление клиента по ID.

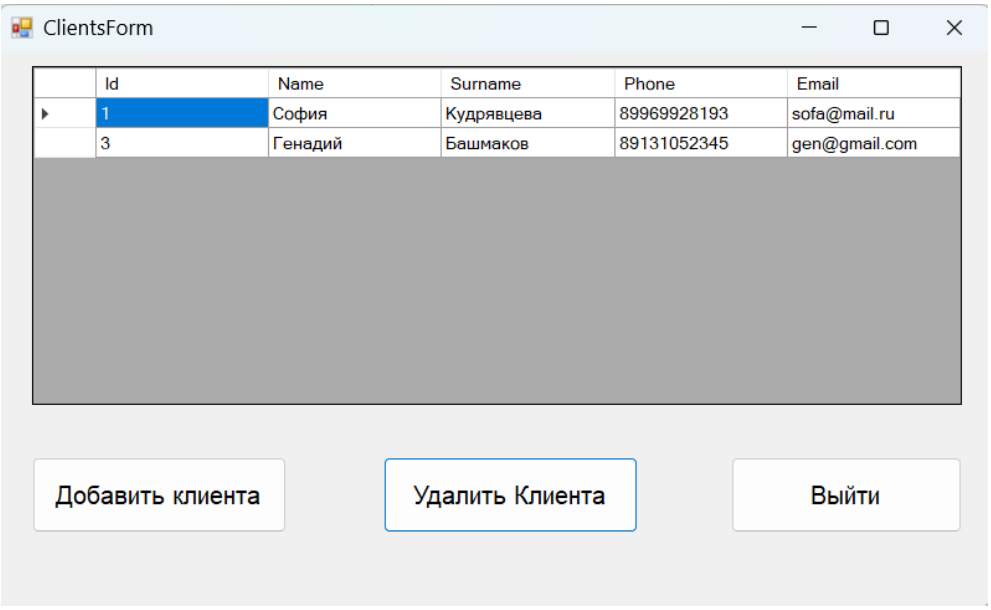

Рисунок 18 – Обновлённая список после удаления.

По кнопке «Выйти» человек попадает обратно на Главную форму.

При нажатии на главной форме кнопки «Список мастеров» происходят процедуры аналогичные списку клиентов, разве что пользователь вводит при добавление так же и тип мастера(Парикмахер, Маникюрщик или Педикюрщик).

При нажатии на главной форме кнопки «Список записи» происходят в целом аналогичные процедуры. Разве что при добавлении записи мы указываем другую информацию, а именно: ID клиента, ID мастера, Дата и время и тип процедуры, рисунок 19. Однако в списке также присутствует категория цена, но она в свою очередь рассчитывается внутри программы по типу процедуры, рисунок 20.

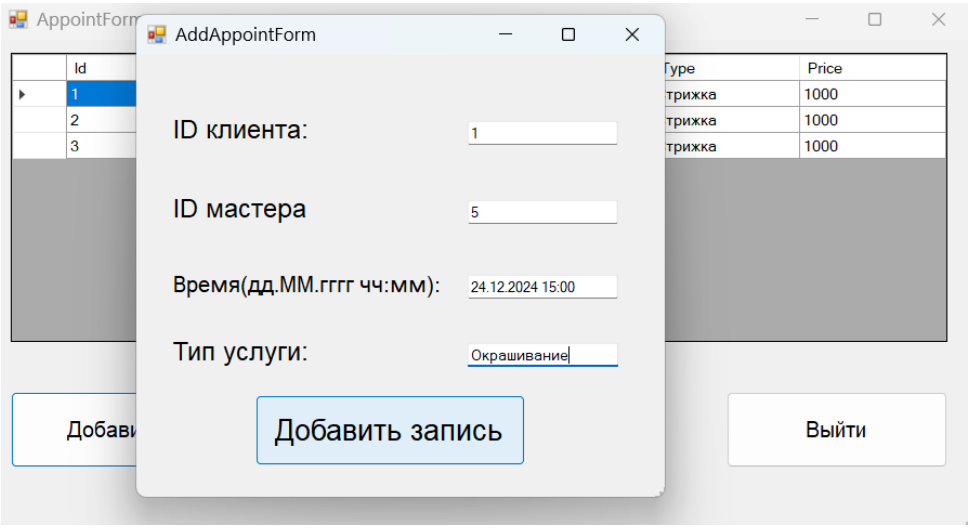

Рисунок 19 – Форма добавления записи.

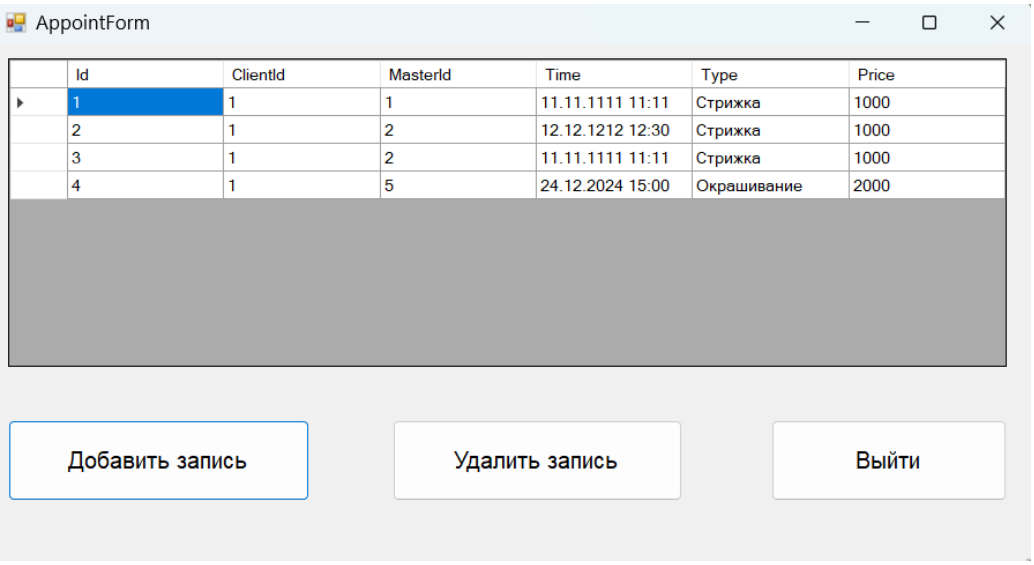

Рисунок 20 – Обновлённый список записей.

5. Заключение

В ходе выполнения проекта по разработке информационной системы для салона красоты были достигнуты поставленные цели и реализованы основные функциональные модули. Система разработана с учетом потребностей современного бизнеса, обеспечивает удобное управление данными клиентов, мастеров и записей на услуги, а также поддерживает сохранение и загрузку данных в формате JSON. Реализация базовых классов и коллекций, таких как Client, Master, Appointment, ClientCol и т.д., и соответствующих им менеджера данных, позволяет организовать эффективное хранение и обработку информации.

Проект достиг своей основной цели — создания информационной системы, которая позволяет автоматизировать процессы управления в салоне красоты, повышая их эффективность и удобство для пользователей. Система может использоваться для работы с реальными данными, а ее структура и модульность облегчают дальнейшее расширение и доработку.

Несмотря на достигнутые результаты, в проекте остается потенциал для улучшений. В будущем возможно добавить больше функциональных возможности, таких как: добавление списка записей к объектам клиентов и мастеров, поиск клиентов, мастеров и записей по таблице, добавление новых процедур со стороны пользователя в приложении и так далее.

Список литературы

1) Сайт салона красоты [электронный ресурс]. – Режим доступа: https://libertynn.ru/ (Дата обращения: 15.12.2024)

2) Полное руководство по языку программирования C# 13 и платформе .NET 9 [электронный ресурс]. – Режим доступа: https://metanit.com/sharp/tutorial/ (Дата обращения: 01.12.2024)

3) Руководство по программированию в Windows Forms [электронный ресурс]. – Режим доступа: https://metanit.com/sharp/windowsforms/ (Дата обращения: 25.12.2024)

4) Официальная документация Microsoft Learn Challenge [электронный ресурс]. – Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/api/system.windows.forms.datagridview?view=windowsdesktop-8.0 (Дата обращения: 12.12.2024)

5) Логанов, С.В. Объектно-ориентированные принципы разработки ИС: учеб. пособие / С.В. Логанов, С.Л. Моругин; Нижегород. гос. техн. ун-т. - Н. Новгород, 2020. - 201 с.

6) Ресурс для публикации статей по программированию Хабр [электронный ресурс]. – Режим доступа: https://habr.com/ru/feed/

Приложения

Salon.cs

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace Salon2
{
    public class Salon
    {
        private ClientCol clientCol;
        private MasterCol masterCol;
        private AppointCol appointCol;
        private dbManager _dbManager;

        public Salon()
        {
            _dbManager = new dbManager();

            // Загрузка данных из базы в коллекции
            clientCol = new ClientCol(_dbManager.LoadClients());
            masterCol = new MasterCol(_dbManager.LoadMasters());
            appointCol = new AppointCol(_dbManager.LoadAppointments());

            Console.WriteLine("Данные из базы успешно загружены.");
        }


        public void addClient(string name, string surname, string phone, string
email)
        {
            // Создаем объект клиента
            Client newClient = new Client(clientCol.GenerateNewId(), name, surname,
phone, email);

            // Добавляем клиента в коллекцию
            clientCol.AddClient(newClient);

            Console.WriteLine($"Клиент {name} {surname} успешно добавлен.");
        }

        public void RemoveClient(int clientId)
        {
            // Найти клиента по ID
            Client clientToRemove = clientCol.GetClients().FirstOrDefault(c => c.Id
== clientId);

            if (clientToRemove == null)
            {
                Console.WriteLine($"Клиент с ID {clientId} не найден.");
                return;
            }

            // Удалить клиента из коллекции
            clientCol.RemoveClient(clientToRemove);

            Console.WriteLine($"Клиент {clientToRemove.Name}
{clientToRemove.Surname} успешно удалён.");
        }
```

```
        public Client GetClientByName(string clientName)
        {
            // Ищем мастера по Name в коллекции мастеров
            var master = clientCol.GetClients().FirstOrDefault(m => m.Name ==
clientName);

            Console.WriteLine($"Мастер с ID {clientName} не найден или не является
парикмахером.");
            return null;
        }


        public List<Client> GetClientCollection()
        {
            return clientCol.GetClients();
        }


        public void addMaster(string name, string surname, string phone, string
email, string type)
        {
            Master newMaster;

            // Создание объекта соответствующего типа
            switch (type.ToLower())
            {
                case "парикмахер":
                    newMaster = new Hairdresser(masterCol.GenerateNewId(), name,
surname, phone, email, type);
                    break;

                case "маникюрщик":
                    newMaster = new Manicurist(masterCol.GenerateNewId(), name,
surname, phone, email, type);
                    break;

                case "педикюрщик":
                    newMaster = new Pedicurist(masterCol.GenerateNewId(), name,
surname, phone, email, type);
                    break;

                default:
                    Console.WriteLine($"Ошибка: Неверный тип мастера '{type}'.
Мастер не добавлен.");
                    return;
            }

            // Добавление мастера в коллекцию
            masterCol.AddMaster(newMaster);
            Console.WriteLine($"Мастер {type} {name} {surname} успешно добавлен в
коллекцию.");
        }

        public void RemoveMaster(int masterId)
        {
            // Найти мастера по ID
            Master masterToRemove = masterCol.GetMasters().FirstOrDefault(m => m.Id
== masterId);

            if (masterToRemove == null)
            {
                Console.WriteLine($"Мастер с ID {masterId} не найден.");
                return;
```

```
            }

            // Удалить мастера из коллекции
            masterCol.RemoveMaster(masterToRemove);

            Console.WriteLine($"Мастер {masterToRemove.Name}
{masterToRemove.Surname} успешно удалён.");
        }

        public Hairdresser GetHairdresserByName(string masterName)
        {
            // Ищем мастера по Name в коллекции мастеров
            var master = masterCol.GetMasters().FirstOrDefault(m => m.Name ==
masterName);

            Console.WriteLine($"Мастер с ID {masterName} не найден или не является
парикмахером.");
            return null;
        }

        public List<Master> GetMasterCollection()
        {
            return masterCol.GetMasters();
        }

        public void AddAppointment(int clientId, int masterId, string dateTime,
string serviceType)
        {
            var client = clientCol.GetClients().FirstOrDefault(c => c.Id ==
clientId);
            var master = masterCol.GetMasters().FirstOrDefault(m => m.Id ==
masterId);
            if (master != null)
            {
                var masterType = master.Type;
            }

            Dictionary<string, int> servicePrices = master.GetServices() ?? new
Dictionary<string, int>(); ;

            if (client == null)
            {
                Console.WriteLine($"Клиент с ID {clientId} не найден.");
                return;
            }

            if (master == null)
            {
                Console.WriteLine($"Мастер с ID {masterId} не найден.");
                return;
            }

            int newId = appointCol.GetAppointments().Count > 0 ?
appointCol.GetAppointments().Max(a => a.Id) + 1 : 1;
            var appointment = new Appointment(appointCol.GenerateNewId(), clientId,
masterId, dateTime, serviceType, servicePrices);
            appointCol.AddAppointment(appointment);
            client.AddAppointment(appointment);
            master.AddAppointment(appointment);
            Console.WriteLine("Запись успешно добавлена.");
        }

        public void RemoveAppointment(int id)
        {
```

```
            appointCol.Remove(id);
            Console.WriteLine($"Запись с ID {id} успешно удалена.");
        }
        public void RemoveAllApp()
        {
            appointCol = new AppointCol();
        }

        public Appointment GetAppByName(int appName)
        {
            // Ищем мастера по ID в коллекции мастеров
            var master = appointCol.GetAppointments().FirstOrDefault(m => m.MasterId
== appName);

            Console.WriteLine($"Мастер с ID {appName} не найден или не является
парикмахером.");
            return null;
        }

        public List<Appointment> GetAllAppointments()
        {
            return appointCol.GetAppointments();
        }


        public void SaveData()
        {
            _dbManager.SaveClients(clientCol.GetClients());
            _dbManager.SaveMasters(masterCol.GetMasters());
            _dbManager.SaveAppointments(appointCol.GetAppointments());
            Console.WriteLine("Данные успешно сохранены в базу.");
        }

        public void ClearClients()
        {
            clientCol.ClearClients();
        }

        public void ClearMasters()
        {
            masterCol.ClearMasters();
        }

    }
}
```

dbManager.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using Newtonsoft.Json;

namespace Salon2
{
    public class dbManager
    {
        private const string ClientDbPath = "clients.json";

        public void SaveClients(List<Client> clients)
        {
            ClearDb(ClientDbPath);
```

```
            string json = JsonConvert.SerializeObject(clients, Formatting.Indented);
            File.WriteAllText(ClientDbPath, json);
        }

        public List<Client> LoadClients()
        {
            if (!File.Exists(ClientDbPath))
                return new List<Client>();

            string json = File.ReadAllText(ClientDbPath);
            return JsonConvert.DeserializeObject<List<Client>>(json) ?? new
List<Client>();
        }

        private const string MasterDbPath = "masters.json";

        public List<Master> LoadMasters()
        {
            if (!File.Exists(MasterDbPath)) return new List<Master>();

            string json = File.ReadAllText(MasterDbPath);

            // Чтение данных как списка RawMaster
            var rawMasters = JsonConvert.DeserializeObject<List<RawMaster>>(json) ??
new List<RawMaster>();
            var masters = new List<Master>();

            foreach (var raw in rawMasters)
            {
                Master master = null;

                // Условный выбор типа мастера
                if (raw.Type == "Hairdresser")
                {
                    master = new Hairdresser(raw.Id, raw.Name, raw.Surname,
raw.Phone, raw.Email, raw.Type);
                }
                else if (raw.Type == "Manicurist")
                {
                    master = new Manicurist(raw.Id, raw.Name, raw.Surname,
raw.Phone, raw.Email, raw.Type);
                }
                else if (raw.Type == "Pedicurist")
                {
                    master = new Pedicurist(raw.Id, raw.Name, raw.Surname,
raw.Phone, raw.Email, raw.Type);
                }

                if (master != null)
                {
                    masters.Add(master);
                }
            }

            return masters;
        }

        public void SaveMasters(List<Master> masters)
        {
            // Сериализация с сохранением типа
            var rawMasters = masters.ConvertAll(m => new RawMaster
            {
                Id = m.Id,
                Name = m.Name,
```

```
                Surname = m.Surname,
                Phone = m.Phone,
                Email = m.Email,
                Type = m.GetType().Name // Тип мастера
            });

            string json = JsonConvert.SerializeObject(rawMasters,
Formatting.Indented);
            File.WriteAllText(MasterDbPath, json);
        }

        private class RawMaster
        {
            public int Id { get; set; }
            public string Name { get; set; }
            public string Surname { get; set; }
            public string Phone { get; set; }
            public string Email { get; set; }
            public Dictionary<string, int> Services { get; set; }
            public string Type { get; set; }
        }

        private const string AppointmentsDbPath = "appointments.json";

        public void SaveAppointments(List<Appointment> appointments)
        {
            // Сериализация данных
            string json = JsonConvert.SerializeObject(appointments,
Formatting.Indented);
            File.WriteAllText(AppointmentsDbPath, json);
        }

        public List<Appointment> LoadAppointments()
        {
            if (!File.Exists(AppointmentsDbPath))
                return new List<Appointment>();

            string json = File.ReadAllText(AppointmentsDbPath);
            return JsonConvert.DeserializeObject<List<Appointment>>(json) ?? new
List<Appointment>();
        }

        public void ClearDb(string filePath)
        {
            if (File.Exists(filePath))
            {
                File.WriteAllText(filePath, string.Empty); // Очищаем файл
            }
            else
            {
                Console.WriteLine($"Файл по пути {filePath} не существует.");
            }
        }

    }
}
```

Client.cs

```
using System.Collections.Generic;
```

```
namespace Salon2
{
    public class Client
    {
        private int _id;
        private string _name;
        private string _surname;
        private string _phone;
        private string _email;
        private List<Appointment> _appointments;

        public int Id { get { return _id; } set { _id = value; } }
        public string Name { get { return _name; } set { _name = value; } }
        public string Surname { get { return _surname; } set { _surname = value; } }
        public string Phone { get { return _phone; } set { _phone = value; } }
        public string Email { get { return _email; } set { _email = value; } }

        public Client()
        {
            _appointments = new List<Appointment>();
        }

        public Client(int id, string name, string surname, string phone, string
email)
        {
            Id = id;
            Name = name;
            Surname = surname;
            Phone = phone;
            Email = email;
        }

        public void AddAppointment(Appointment appointment)
        {
            _appointments.Add(appointment);
        }

        public List<Appointment> GetAppointments()
        {
            return _appointments;
        }

        public override string ToString()
        {
            return $"ID: {Id}, Name: {Name}, Surname: {Surname}, Phone: {Phone},
Email: {Email}";
        }
    }

}
```

ClientCol.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Salon2
{
    public class ClientCol
```

```
    {
        private List<Client> _clients;

        public ClientCol()
        {
            _clients = new List<Client>();
        }

        public ClientCol(List<Client> clients)
        {
            _clients = clients ?? new List<Client>();
        }

        /// <summary>
        /// Добавляет клиента в коллекцию.
        /// </summary>
        public void AddClient(Client client)
        {
            _clients.Add(client);
        }

        /// <summary>
        /// Генерирует новый ID для клиента.
        /// </summary>
        public int GenerateNewId()
        {
            if (_clients == null || _clients.Count == 0)
            {
                return 1; // Если коллекция пуста, начинаем с ID = 1
            }
            return _clients.Max(c => c.Id) + 1; // Генерация нового ID на основе
максимального
        }

        public void RemoveClient(Client client)
        {
            _clients.Remove(client);
        }

        /// <summary>
        /// Получает список всех клиентов.
        /// </summary>
        public List<Client> GetClients()
        {
            return _clients;
        }

        /// <summary>
        /// Очищает коллекцию клиентов.
        /// </summary>
        public void ClearClients()
        {
            _clients.Clear();
            Console.WriteLine("Все клиенты были удалены из коллекции.");
        }
    }

}
```

Master.cs

```
using System;
```

```
using System.Collections.Generic;
using System.Linq;
using System.Runtime.InteropServices;
using System.Security.Cryptography.X509Certificates;
using System.Text;
using System.Threading.Tasks;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.ListView;
using System.Xml.Linq;

namespace Salon2
{
    public class Master
    {
        private int _id;
        private string _name;
        private string _surname;
        private string _phone;
        private string _email;
        private List<Appointment> _appointments;
        private string _type;

        public int Id { get { return _id; } set { _id = value; } }
        public string Name { get { return _name; } set { _name = value; } }
        public string Surname { get { return _surname; } set { _surname = value; } }
        public string Phone { get { return _phone; } set { _phone = value; } }
        public string Email { get { return _email; } set { _email = value; } }

        public string Type { get { return _type; } set { _type = value; } }

        public Master(int id, string name, string surname, string phone, string
email, string type)
        {
            Id = id;
            Name = name;
            Surname = surname;
            Phone = phone;
            Email = email;
            Type = type;
            _appointments = new List<Appointment>();
        }

        public void AddAppointment(Appointment appointment)
        {
            _appointments.Add(appointment);
        }

        public List<Appointment> GetAppointments()
        {
            return _appointments;
        }

        public Dictionary<string,int> GetServices()
        {
            switch (Type)
            {
                case "Hairdresser":
                    {
                        Hairdresser master = new Hairdresser(Id, Name, Surname,
Phone, Email, Type);
                        return master.Getserv();
                    }
                case "Manicurist":
                    {
```

```
                        Hairdresser master = new Hairdresser(Id, Name, Surname,
Phone, Email, Type);
                        return master.Getserv();
                    }
                case "Pedicurist":
                    {
                        Hairdresser master = new Hairdresser(Id, Name, Surname,
Phone, Email, Type);
                        return master.Getserv();
                    }
            }
            return null;
        }
    }
}
```

Hairdresser.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Salon2
{
    public class Hairdresser : Master
    {

        public Dictionary<string, int> Services = new Dictionary<string, int> { };

        public Hairdresser(int id, string name, string surname, string phone, string
email, string type)
            : base(id, name, surname, phone, email, type/*, services*/)
        {
            // Определяем услуги и их стоимость
            Services = new Dictionary<string, int>
            {
                { "Стрижка", 1000 },
                { "Окрашивание", 2000 }
            };
        }

        public Dictionary<string,int> Getserv()
        {
            return Services;
        }
    }
}
```

Manicurist.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Salon2
{
```

```
    public class Manicurist : Master
    {
        public Dictionary<string, int> Services = new Dictionary<string, int> { };

        public Manicurist(int id, string name, string surname, string phone, string
email, string type)
            : base(id, name, surname, phone, email, type)
        {
            // Определяем услуги и их стоимость
            Services = new Dictionary<string, int>
            {
                { "Наращивание", 1200 },
                { "Дизайн", 800 }
            };

        }

        public Dictionary<string, int> Getserv()
        {
            return Services;
        }
    }
}
```

Pedicurist.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Salon2
{
    public class Pedicurist : Master
    {
        public Dictionary<string, int> Services = new Dictionary<string, int> { };

        public Pedicurist(int id, string name, string surname, string phone, string
email, string type)
            : base(id, name, surname, phone, email, type/*, services*/)
        {
            // Определяем услуги и их стоимость
            Services = new Dictionary<string, int>
            {
                { "Педикюр", 900 },
                { "Гелевый педикюр", 1200 }
            };
        }

        public Dictionary<string, int> Getserv()
        {
            return Services;
        }
    }
}
```

MasterCol.cs

```
using System;
```

```
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Salon2
{
    public class MasterCol
    {
        private List<Master> _masters;

        public MasterCol()
        {
            _masters = new List<Master>();
        }

        public MasterCol(List<Master> masters)
        {
            _masters = masters ?? new List<Master>();
        }

        public int GenerateNewId()
        {
            if (_masters == null || _masters.Count == 0)
            {
                return 1; // Если коллекция пуста, начинаем с ID = 1
            }
            return _masters.Max(c => c.Id) + 1; // Генерация нового ID на основе
максимального
        }

        public void AddMaster(Master master)
        {
            _masters.Add(master);
        }

        public void RemoveMaster(Master master)
        {
            _masters.Remove(master);
        }

        public List<Master> GetMasters()
        {
            return _masters;
        }

        public void ClearMasters()
        {
            _masters.Clear();
            Console.WriteLine("Все клиенты были удалены из коллекции.");
        }
    }
}
```

Appointment.cs

```
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;
```

```
using System.Text;
using System.Threading.Tasks;

namespace Salon2
{
    public class Appointment
    {
        private int _id;
        private int _clientid;
        private int _masterid;
        private string _datetime;
        private string _type;
        private int _price;
        public int Id { get { return _id; } set { _id = value; } }
        public int ClientId { get { return _clientid; } set { _clientid = value; } }
        public int MasterId { get { return _masterid; } set { _masterid = value; } }
        public string Time { get { return _datetime; } set { _datetime = value; } }
        public string Type { get { return _type; } set { _type = value; } }
        public int Price { get { return _price; } set { _price = value; } }

        public Appointment(int id, int clientId, int masterId, string
appointmentDateTime, string serviceType, Dictionary<string,int> servicePrices)
        {
            string format = "dd.MM.yyyy HH:mm";
            if (appointmentDateTime != null)
            {
                DateTime date = DateTime.ParseExact(appointmentDateTime, format,
CultureInfo.InvariantCulture);
                if (date.Hour < 9 || date.Hour >= 17)
                    throw new ArgumentException("Время записи должно быть в
интервале с 09:00 до 17:00.");
            }


            Id = id;
            ClientId = clientId;
            MasterId = masterId;
            Time = appointmentDateTime;
            Type = serviceType;
            if (servicePrices != null)
            {
                Price = servicePrices[serviceType];
            }
        }

        public override string ToString()
        {
            return $"ID: {Id}, Client ID: {ClientId}, Master ID: {MasterId}, Date &
Time: {Time}";
        }
    }
}
```

AppointCol.cs

```
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
namespace Salon2
{
    public class AppointCol
    {
        private List<Appointment> _appointments;

        public AppointCol()
        {
            _appointments = new List<Appointment>();
        }

        public AppointCol(List<Appointment> appointments)
        {
            _appointments = appointments ?? new List<Appointment>();
        }

        public void AddAppointment(Appointment appointment)
        {
            _appointments.Add(appointment);
        }

        public void Remove(int id)
        {
            var appointment = _appointments.FirstOrDefault(a => a.Id == id);
            if (appointment != null)
                _appointments.Remove(appointment);
            else
                throw new ArgumentException("Запись с указанным ID не найдена.");
        }

        public Appointment GetById(int id)
        {
            return _appointments.FirstOrDefault(a => a.Id == id);
        }

        public List<Appointment> GetAppointments()
        {
            return _appointments;
        }

        public int GenerateNewId()
        {
            if (_appointments == null || _appointments.Count == 0)
            {
                return 1; // Если коллекция пуста, начинаем с ID = 1
            }
            return _appointments.Max(c => c.Id) + 1; // Генерация нового ID на
основе максимального
        }
    }
}
```

MainForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
```

```
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Salon2
{
    public partial class MainForm : Form
    {
        private Salon salon;
        public MainForm()
        {
            salon = new Salon();
            InitializeComponent();

        }

        private void buttonListClients_Click(object sender, EventArgs e)
        {
            this.Hide();
            var clientsForm = new ClientsForm(salon);
            clientsForm.ShowDialog();
            this.Show();
        }

        private void buttonListMasters_Click(object sender, EventArgs e)
        {
            this.Hide();
            var mastersForm = new MastersForm(salon);
            mastersForm.ShowDialog();
            this.Show();

        }

        private void buttonListAppoint_Click(object sender, EventArgs e)
        {
            this.Hide();
            var appointForm = new AppointForm(salon);
            appointForm.ShowDialog();
            this.Show();
        }

        private void MainForm_FormClosed(object sender, FormClosedEventArgs e)
        {
            salon.SaveData();
        }


    }
}
```

AppointForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Salon2
```

```
{
    public partial class AppointForm : Form
    {
        private Salon salon;
        public AppointForm(Salon salon)
        {
            this.salon = salon;
            InitializeComponent();
            LoadAppoints();
        }

        private void LoadAppoints()
        {
            // Загрузка данных клиентов в таблицу
            dataGridView1.DataSource = null;
            dataGridView1.AutoGenerateColumns = true;
            dataGridView1.DataSource = salon.GetAllAppointments();

        }

        private void buttonAddAppoint_Click(object sender, EventArgs e)
        {
            // Открытие формы добавления клиента
            var addAppointForm = new AddAppointForm(salon);
            addAppointForm.ShowDialog();
            LoadAppoints(); // Обновление списка клиентов после добавления
        }

        private void buttonDelAppoint_Click(object sender, EventArgs e)
        {
            var delAppointForm = new DelAppointForm(salon);
            delAppointForm.ShowDialog();
            LoadAppoints();
        }

        private void buttonExit_Click(object sender, EventArgs e)
        {
            this.Close();
        }
    }
}
```

ClientForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Salon2
{
    public partial class ClientsForm : Form
    {
        private Salon salon;

        public ClientsForm(Salon salon)
        {
```

```
            this.salon = salon;
            InitializeComponent();
            LoadClients();
        }

        private void LoadClients()
        {
            // Загрузка данных клиентов в таблицу
            dataGridView1.DataSource = null;
            dataGridView1.AutoGenerateColumns = true;
            dataGridView1.DataSource = salon.GetClientCollection();
        }

        private void buttonAddClient_Click(object sender, EventArgs e)
        {
            // Открытие формы добавления клиента
            var addClientForm = new AddClientForm(salon);
            addClientForm.ShowDialog();
            LoadClients(); // Обновление списка клиентов после добавления
        }

        private void buttonDelClient_Click(object sender, EventArgs e)
        {
            var delClientForm = new DelClientForm(salon);
            delClientForm.ShowDialog();
            LoadClients();
        }

        private void buttonExit_Click(object sender, EventArgs e)
        {
            this.Close();
        }
    }
}
```

MasterForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Salon2
{
    public partial class MastersForm : Form
    {
        private Salon salon;

        public MastersForm(Salon salon)
        {
            this.salon = salon;
            InitializeComponent();
            LoadMasters();
        }

        private void LoadMasters()
        {
```

```
            // Загрузка данных клиентов в таблицу
            dataGridView1.DataSource = null;
            dataGridView1.AutoGenerateColumns = true;
            dataGridView1.DataSource = salon.GetMasterCollection();
        }

        private void buttonAddMaster_Click(object sender, EventArgs e)
        {
            var addMasterForm = new AddMasterForm(salon);
            addMasterForm.ShowDialog();
            LoadMasters();
        }

        private void buttonDelMaster_Click(object sender, EventArgs e)
        {
            var delMasterForm = new DelMasterForm(salon);
            delMasterForm.ShowDialog();
            LoadMasters();
        }

        private void buttonExit_Click(object sender, EventArgs e)
        {
            this.Close();
        }
    }
}
```

AddAppointForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Salon2
{
    public partial class AddAppointForm : Form
    {
        private Salon salon;
        public AddAppointForm(Salon salon)
        {
            this.salon = salon;
            InitializeComponent();
        }

        private void buttonAddAppoint_Click(object sender, EventArgs e)
        {
            // Попытка преобразовать строку из textBoxDate.Text в DateTime
            if (DateTime.TryParseExact(textBoxDate.Text, "dd.MM.yyyy HH:mm",
                System.Globalization.CultureInfo.InvariantCulture,
                System.Globalization.DateTimeStyles.None,
                out DateTime date))
            {
                // Попытка преобразовать textBoxIDC.Text в int
                if (int.TryParse(textBoxIDC.Text, out int idC) &&
                    int.TryParse(textBoxIDM.Text, out int idM))
                {
```

```
                    // Передача корректно преобразованных данных
                    salon.AddAppointment(idC, idM, textBoxDate.Text,
textBoxType.Text);

                    MessageBox.Show("Запись добавлена.", "Успех",
MessageBoxButtons.OK, MessageBoxIcon.Information);
                    this.Close();
                }
                else
                {
                    MessageBox.Show("Некорректный ID клиента или мастера! Введите
целые числа.",
                        "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
                }
            }
            else
            {
                MessageBox.Show("Некорректный формат даты и времени! Используйте
формат 'дд.ММ.гггг чч:мм'",
                    "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
    }
}
```

AddClientForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Xml.Linq;

namespace Salon2
{
    public partial class AddClientForm : Form
    {
        private Salon salon;
        public AddClientForm(Salon salon)
        {
            this.salon = salon;
            InitializeComponent();
        }

        private void AddClient(object sender, EventArgs e)
        {
            // Создание нового клиента и добавление в коллекцию
            salon.addClient(textBoxName.Text, textBoxSurname.Text,
textBoxPhone.Text, textBoxEmail.Text);
            MessageBox.Show("Клиент добавлен.", "Успех", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            this.Close();
        }
    }
}
```

AddMasterForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Salon2
{
    public partial class AddMasterForm : Form
    {
        private Salon salon;
        public AddMasterForm(Salon salon)
        {
            this.salon = salon;
            InitializeComponent();
        }

        private void buttonAddMaster_Click(object sender, EventArgs e)
        {
            // Создание нового клиента и добавление в коллекцию
            salon.addMaster(textBoxName.Text, textBoxSurname.Text,
textBoxPhone.Text, textBoxEmail.Text, textBoxType.Text);
            MessageBox.Show("Мастер добавлен.", "Успех", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            this.Close();
        }
    }
}
```

DelAppointForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace Salon2
{
    public partial class DelAppointForm : Form
    {
        private Salon salon;
        public DelAppointForm(Salon salon)
        {
            this.salon = salon;
            InitializeComponent();
        }

        private void DelAppoint(object sender, EventArgs e)
        {
            int id = int.Parse(textBoxID.Text);
            salon.RemoveAppointment(id);
```

```
            MessageBox.Show("Запись удалёна.", "Успех", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            this.Close();
        }
    }
}
```

DelClientForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Salon2
{
    public partial class DelClientForm : Form
    {
        private Salon salon;
        public DelClientForm(Salon salon)
        {
            this.salon = salon;
            InitializeComponent();
        }

        private void DelClient(object sender, EventArgs e)
        {
            int id = int.Parse(textBoxID.Text);
            salon.RemoveClient(id);
            MessageBox.Show("Клиент удалён.", "Успех", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            this.Close();
        }
    }
}
```

DelMasterForm.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace Salon2
{
    public partial class DelMasterForm : Form
    {
        private Salon salon;
        public DelMasterForm(Salon salon)
```

```
        {
            this.salon = salon;
            InitializeComponent();
        }

        private void buttonDelMaster_Click(object sender, EventArgs e)
        {
            int id = int.Parse(textBoxID.Text);
            salon.RemoveMaster(id);
            MessageBox.Show("Мастер удалён.", "Успех", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            this.Close();
        }
    }
}
```

UnitTest1.cs

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using System;
using System.Collections.Generic;
using Salon2;
using NUnit;

namespace Tests
{
    [TestClass]
    public class UnitTest1
    {

        private Salon _salon;

        [TestInitialize]
        public void Setup()
        {
            _salon = new Salon();
            _salon.ClearClients();
            _salon.ClearMasters();
        }

        [TestMethod]
        public void AddClient_ShouldAddClientToCollection()
        {
            // Arrange

            string name = "John";
            string surname = "Doe";
            string phone = "1234567890";
            string email = "john.doe@example.com";

            // Act
            _salon.addClient(name, surname, phone, email);

            // Assert
            List<Client> clients = _salon.GetClientCollection();
            Assert.AreEqual(1, clients.Count);
            Assert.AreEqual(1, clients[0].Id);
            Assert.AreEqual(name, clients[0].Name);
            Assert.AreEqual(surname, clients[0].Surname);
```

```
}

[TestMethod]
public void RemoveClient_ShouldRemoveClientFromCollection()
{
    // Arrange
    string name = "John";
    string surname = "Doe";
    string phone = "1234567890";
    string email = "john.doe@example.com";
    _salon.addClient(name, surname, phone, email);

    int clientId = _salon.GetClientCollection()[0].Id;

    // Act
    _salon.RemoveClient(clientId);

    // Assert
    List<Client> clients = _salon.GetClientCollection();
    Assert.AreEqual(0, clients.Count);
}

[TestMethod]
public void AddMaster_ShouldAddMasterToCollection()
{
    // Arrange
    string name = "Jane";
    string surname = "Smith";
    string phone = "0987654321";
    string email = "jane.smith@example.com";
    string type = "парикмахер";

    // Act
    _salon.addMaster(name, surname, phone, email, type);

    // Assert
    List<Master> masters = _salon.GetMasterCollection();
    Assert.AreEqual(1, masters.Count);
    Assert.AreEqual(name, masters[0].Name);
    Assert.AreEqual(surname, masters[0].Surname);
}

[TestMethod]
public void RemoveMaster_ShouldRemoveMasterFromCollection()
{
    // Arrange
    string name = "Jane";
    string surname = "Smith";
    string phone = "0987654321";
    string email = "jane.smith@example.com";
    string type = "парикмахер";
    _salon.addMaster(name, surname, phone, email, type);

    int masterId = _salon.GetMasterCollection()[0].Id;

    // Act
    _salon.RemoveMaster(masterId);

    // Assert
    List<Master> masters = _salon.GetMasterCollection();
    Assert.AreEqual(0, masters.Count);
}

[TestMethod]
```

```
        public void SaveData_ShouldSaveClientsAndMastersToDatabase()
        {
            // Arrange
            string clientName = "John";
            string clientSurname = "Doe";
            string clientPhone = "1234567890";
            string clientEmail = "john.doe@example.com";
            _salon.addClient(clientName, clientSurname, clientPhone, clientEmail);

            string masterName = "Jane";
            string masterSurname = "Smith";
            string masterPhone = "0987654321";
            string masterEmail = "jane.smith@example.com";
            string type = "парикмахер";
            _salon.addMaster(masterName, masterSurname, masterPhone, masterEmail,
type);

            // Act
            _salon.SaveData();

            // Assert
            var dbManager = new dbManager();
            List<Client> savedClients = dbManager.LoadClients();
            List<Master> savedMasters = dbManager.LoadMasters();

            Assert.AreEqual(1, savedClients.Count);
            Assert.AreEqual(clientName, savedClients[0].Name);
            Assert.AreEqual(1, savedMasters.Count);
            Assert.AreEqual(masterName, savedMasters[0].Name);
        }

    }
}
```